

REGIONAL AGRICULTURAL AI - Phase 1

Part 1: Environment and Project Setup

1. Project Introduction

The Regional Agricultural AI project aims to build an AI-based crop leaf disease detection system that analyzes a crop leaf image and identifies the disease with a confidence score.

Phase 1 focuses on the Core Machine Learning Disease Prediction System, isolating the model development from web applications, APIs, and deployments (which will follow in later phases).

2. Part Objective: Environment and Project Setup

The goal of this part is to create a robust, reproducible Machine Learning project environment. A clean environment prevents dependency conflicts, ensures that code runs smoothly on different machines, and creates a logical structure for all scripts and data.

3. Project Setup Concepts

- Python Version: We are using Python 3, which is the standard for ML development due to its rich ecosystem.
- Virtual Environment (venv): An isolated environment that contains its own Python executable and installed packages. It prevents global packages from interfering with project-specific dependencies.
- pip: Python's package installer, used to download and install libraries from the Python Package Index (PyPI).
- requirements.txt: A file listing all the packages required to run the project. This makes the project reproducible; anyone can run `pip install -r requirements.txt` to replicate the environment.
- Jupyter Notebook: An interactive computational environment used for data exploration, visualization, and step-by-step experimentation.
- VS Code: A powerful source-code editor where the final Python scripts will be developed.
- GPU vs CPU / CUDA: Deep learning models (like CNNs) train much faster on a GPU. CUDA is a parallel computing platform by NVIDIA that enables TensorFlow/Keras to leverage the GPU for rapid matrix operations. If no GPU is available, the CPU is used (which is much slower).

4. Project Architecture

The workspace has been organized as follows:

- regional-agricultural-ai/
- dataset/ (Original and processed datasets)
- notebooks/ (Jupyter notebooks)
- src/ (Source code)
- data/ (Data ingestion)
- preprocessing/ (Cleaning and transforms)

- models/ (Architectures)
- training/ (Training loops)
- evaluation/ (Metrics)
- inference/ (Prediction scripts)
- models/ (Saved models)
- results/ (Training logs)
- reports/ (Generated docs)
- requirements.txt (Dependencies)

5. Technologies & Packages

- numpy: Fundamental package for scientific computing. Required for multi-dimensional arrays (tensors).
- pandas: Data manipulation and analysis tool. Required for handling tabular data.
- matplotlib & seaborn: Plotting libraries. Required for generating graphs.
- opencv-python: Open Source Computer Vision Library. Required for fast image reading, resizing.
- Pillow: Python Imaging Library. Required as a secondary image processing tool.
- scikit-learn: Machine learning library. Required for creating train/validation/test splits, calculating metrics.
- tensorflow & keras: The core deep learning framework. (Note: Awaits Python 3.11/3.12 or conda environment).
- tqdm: A smart progress meter. Required for showing progress bars.
- jupyter & notebook: Required to run .ipynb files for exploratory data analysis.

6. Code Explanation (Setup)

Command: `python -m venv venv`

Explanation: Creates a folder named 'venv' containing the isolated Python environment.

Command: `.\venv\Scripts\activate`

Explanation: Activates the environment so that subsequent commands use the isolated pip and Python.

Command: `pip install -r requirements.txt`

Explanation: Installs all listed dependencies automatically.

7. Viva & Interview Questions

Q: Why do we use a virtual environment?

A: To isolate project dependencies from the global system, ensuring reproducibility and preventing version conflicts.

Q: What happens if you don't use requirements.txt?

A: Another developer (or the examiner) won't know which packages and versions to install, potentially causing the code to crash.

Q: Why separate data processing code from model training code?

A: It makes the codebase modular, easier to test, debug, and maintain.

8. Implementation Status

Environment and Project Setup: COMPLETED.

The dependencies are installed, folders are created, and this document proves the successful execution of Part 1.